Week 2 Day 1 — Fetch a URL's Text (Oct 14, 2026)

What was built

`web_fetch.py`: `fetch_url(url, timeout=15) -> dict` — the agent's first real tool. Real HTTP fetch via `httpx`, text extraction with `html.parser` (a small `_TextExtractor` that skips script and style content and collapses whitespace), and a 50KB cap on returned text so a huge page can't eat the model's context. Returns clean text on success, or a structured error dict on failure: `timeout`, `request_error`, `http_404`, `http_error`, `non_html`.

`tools.py`: the Week 2 registry, same dispatch shape as Week 1 — a `TOOLS` dict mapping name to function, and `run_tool(name, arg)` routing one string argument. It starts with one tool today and gains the shell and file tools tomorrow.

Done criteria (all met): `fetch_url` returns clean text for a real page · a 404 and a non-HTML response each return a structured error observation, not an exception · all three verified by running them through the registry. Committed as `d51285e`.

What actually happened

The tool took twenty minutes. The environment took the rest of the morning, and every detour taught something real.

Dedicated env. Created a `agent-sprint` conda env (Python 3.12.15) instead of pip-installing into base. The sprint's dependencies are the project's, and the repo can recreate itself from scratch. `httpx` and `keyring` went into the new env. One gotcha carried over: the gateway key lives in Windows Credential Manager, not in any env, so it survived the move — but the `keyring` package itself needed reinstalling.

VS Code. The new env didn't show in the interpreter picker; fixed by pasting the full interpreter path. The integrated terminal then couldn't find `python` or `conda` at all — PowerShell didn't have conda on PATH, and `cd D:\agent-sprint` silently did nothing because `cd` doesn't switch drives in cmd. Fixes: `conda init powershell` (one-time, new terminals only), and `D:` then `cd agent-sprint` to actually change drives.

Zscaler. The first fetch failed with `SSL: CERTIFICATE_VERIFY_FAILED`. Diagnosis, in order: certifi's bundle existed; `SSL_CERT_FILE` pointed at the conda bundle, which existed too (189KB, valid header); Python's cert read came back empty; openssl showed the real story — the certificate for `example.com` was issued by `Zscaler Intermediate Root CA`, verify return code 20. Zscaler, the corporate proxy, re-signs every HTTPS connection. Browsers trust it because its root is in the Windows store; Python uses its own bundle, which had no Zscaler root. Fix: exported the Zscaler root CA from the Windows store and appended it to the env's `cacert.pem`. Verification stays fully on. No new trust was added — Python now trusts exactly what Windows already trusts. `verify=False` was explicitly not used.

Test URL lesson. The W3C dummy PDF link 404'd, so the tool correctly reported a dead link instead of exercising the `non_html` path. Switched to the IRS W-9 PDF, which returned `non_html` as expected. A test that fails for the wrong reason is still information.

The core insight

The Day 1 lesson landed before the tool was even built: unhappy paths come back as observations, not exceptions. The Zscaler failure was the first live demo — the tool returned a clean error dict instead of crashing, and the diagnosis happened through the observation. Second insight, same morning: the environment is part of the system. The code was right; the trust store was the bug.

Line-by-line walkthrough

```
response = httpx.get(url, timeout=timeout, follow_redirects=True)
```

One call, three behaviors baked in: a timeout so a hung server can't hang the agent, and redirect-following so a moved page still resolves. The timeout value (15s default) is a design choice, not a default to ignore — it bounds how long one tool call can stall the loop.

```
except httpx.TimeoutException as e:
    return {"ok": False, "error": "timeout", "detail": str(e)}
except httpx.RequestError as e:
    return {"ok": False, "error": "request_error", "detail": str(e)}
```

Two except clauses, specific before general. Timeout gets its own error code because the loop's response differs: a timeout might be worth retrying, a DNS failure probably isn't. Both return dicts, never raise — that's the contract.

```
if response.status_code == 404:
    return {"ok": False, "error": "http_404", ...}
if not 200 <= response.status_code < 300:
    return {"ok": False, "error": "http_error", ...}
```

404 gets its own code because it's the most common real-world failure and the most actionable (the link is dead, try another). Everything else non-2xx collapses to `http_error` with the status in the detail.

```
content_type = response.headers.get("content-type", "")
if "text/html" not in content_type:
    return {"ok": False, "error": "non_html", ...}
```

The content-type check is what keeps a PDF from becoming garbage text in the prompt. Note the `""` default: a missing header is treated as non-HTML rather than trusted.

```
extractor = _TextExtractor()
extractor.feed(response.text)
text = extractor.text()[:MAX_CHARS]
```

The `_TextExtractor` keeps a skip-depth counter: entering a `<script>` or `<style>` tag increments it, leaving decrements it, and text is collected only at depth zero. The `[:MAX_CHARS]` slice is the context budget — 50KB of text is the most any one fetch can cost the model.